

```

float f;
va_list ptr;

va_start ( ptr, num );
printf ( "\n" );
switch ( type )
{
    case 1:
        for ( j = 1 ; j <= num ; j++ )
        {
            i = va_arg ( ptr, int );
            printf ( "%d ", i );
        }
        break ;

    case 2:
        for ( j = 1 ; j <= num ; j++ )
        {
            c = va_arg ( ptr, char );
            printf ( "%c ", c );
        }
        break ;

    case 3:
        for ( j = 1 ; j <= num ; j++ )
        {
            f = ( float ) va_arg ( ptr, double );
            printf ( "%f ", f );
        }
    }
}

```

Here we pass two fixed arguments to the function **display()**. The first one indicates the data type of the arguments to be printed and the second indicates the number of such arguments to be printed. Once again, through the statement **va_start (ptr, num)** we set up **ptr** such that it points to the first argument in the variable list of

arguments. Then depending upon whether the value of **type** is 1, 2 or 3, we print out the arguments as **ints**, **chars** or **floats**.

In all calls to **display()** the second argument indicated how many values are we trying to print. Contrast this with **printf()**. To it we never pass an argument indicating how many value are we trying to print. Then how does **printf()** figure this out? Simple. It scans the format string and counts the number of format specifiers that we have used in it to decide how many values are being printed.

Unions

Unions are derived data types, the way structures are. But Unions have the same relationship to structures that you might have with a distant cousin who resembled you but turned out to be smuggling contraband in Mexico. That is, unions and structures look alike, but are engaged in totally different enterprises.

Both structures and unions are used to group a number of different variables together. But while a structure enables us treat a number of different variables stored at different places in memory, a union enables us to treat the same space in memory as a number of different variables. That is, a union offers a way for a section of memory to be treated as a variable of one type on one occasion, and as a different variable of a different type on another occasion.

You might wonder why it would be necessary to do such a thing, but we will be seeing several very practical applications of unions soon. First, let us take a look at a simple example:

```

/* Demo of union at work */
#include <stdio.h>
void main()
{
    union a
    {

```